

C++基础： 指针和链表

目录

- 什么是指针
 - 内存地址
 - 声明指针
 - 解引用操作符
- 指针的用途
 - 动态内存分配
 - 传递参数
 - 使用指针访问数组
- 指针的高级用法
 - 指针算术
 - 指向指针的指针
 - `const`指针和指向常量的指针
- 内存管理和指针
 - 内存泄漏和内存泄漏检测
 - `new`和`delete`操作符

什么是链表

链表的基本概念

链表节点的结构

单向链表

单向链表的操作

链表遍历

链表的插入和删除操作

双向链表

双向链表的操作

双向链表的插入和删除操作

链表的高级用法

循环链表

带头结点的链表

带环链表

链表的排序和搜索

什么是指针

- 指针是一个变量，它存储了一个地址，这个地址指向内存中的某个位置。指针的定义方式为：类型* 变量名；其中类型表示指针指向的数据类型，变量名为指针变量的名称。例如，`int* p;` 就定义了一个指向整数的指针变量p。
- 指针的物理意义是指，指针变量中存储的是内存地址。每个变量在内存中都有一个唯一的地址，指针变量中存储的地址就是该指针所指向的变量在内存中的地址。可以使用`&`运算符来获取一个变量的地址，例如`&x`就表示变量x在内存中的地址。
- 指针的逻辑意义是指，它提供了一种间接访问内存中数据的方法。通过指针变量中存储的地址，可以访问该地址处存储的数据。例如，如果有一个整数变量x，我们可以使用指针p来访问x的值，即`*p`表示指向p所指向地址处的整数值。

什么是指针

- 假设我们有一个整数变量 x ，它的值为10，我们想要通过指针访问它的值。

```
#include <iostream>
using namespace std;

int main() {
    int x = 10;
    int *p = &x; // p指向x的地址
    cout << "x的值为：" << x << endl;
    cout << "p所指向的地址为：" << p << endl;
    cout << "p所指向的值为：" << *p << endl;
    return 0;
}
```

```
x的值为：10
p所指向的地址为：0x7ffee70b1a7c
p所指向的值为：10
```

- 从运行结果可以看出，我们首先定义了一个整数变量 x 并赋值为10，然后定义了一个指向整数的指针变量 p ，并将其赋值为 x 的地址（ $\&x$ 表示 x 在内存中的地址）。接着使用 $*p$ 来访问 p 所指向地址处的值，即 x 的值，输出了 x 的值和 p 所指向的地址和值。

内存地址

- 在C/C++中，指针是一个非常重要的概念，它提供了一种间接访问内存中数据的方法。指针实际上就是内存中的一个地址，指向内存中的某个位置，可以通过指针来访问该位置中存储的数据。**指针的核心概念就是内存地址。**
- 每个变量在内存中都有一个唯一的地址，这个地址用来表示该变量在内存中的位置。例如，假设有一个整数变量`x`，它的值为`10`，那么在内存中它的位置就是一个地址，例如`0x7fff5fbff7d4`。

声明指针

- 在C/C++中，我们可以使用指针来访问内存中的数据，通过声明指针变量并赋予它一个地址来创建一个指针。指针变量的声明方式如下：

```
int *p: //声明一个指向整数的指针变量p
```

- 这个声明语句表示创建了一个指向整数的指针变量p。需要注意的是，指针变量的类型必须与它所指向的数据类型相同。

```
#include <iostream>
using namespace std;

int main() {
    int x = 10;
    int *p: // 声明指针变量p
    p = &x; // 把变量x的地址赋值给p
    return 0;
}
```

- 上述代码中，我们首先定义了一个整数变量x并赋值为10，然后声明了一个指向整数的指针变量p，将变量x的地址赋值给了p。此时，指针变量p就指向了变量x所在的内存地址。

解引用操作符

- 上述代码中，我们首先定义了一个整数变量`x`并赋值为10，然后声明了一个指向整数的指针变量`p`，将变量`x`的地址赋值给了`p`。此时，指针变量`p`就指向了变量`x`所在的内存地址。

```
#include <iostream>
using namespace std;

int main() {
    int x = 10;
    int *p;    // 声明指针变量p
    p = &x;    // 把变量x的地址赋值给p
    cout << "x的值为: " << *p << endl; // 输出x的值
    return 0;
}
```

- 使用解引用操作符来输出指针所指向的变量的值。由于`p`指向变量`x`的地址，因此`p`就表示变量`x`的值，上述代码会输出`x`的值为10。

解引用操作符

- 除了可以使用***p**来访问指针指向的变量的值之外，还可以通过给指针变量赋值来改变变量的值。例如，下面的代码演示了如何通过指针来改变变量的值：

```
#include <iostream>
using namespace std;

int main() {
    int x = 10;
    int *p;    // 声明指针变量p
    p = &x;    // 把变量x的地址赋值给p
    *p = 20;   // 改变变量x的值
    cout << "x的新值为: " << x << endl; // 输出x的新值
    return 0;
}
```

- 上述代码中，我们将指针**p**所指向的变量的值改为了20，此时输出变量**x**的值就变成了20。

什么是指针

- 指针是C/C++中一个非常重要的概念：
 - 它提供了一种间接访问内存中数据的方法。
 - 指针实际上就是内存中的一个地址，指向内存中的某个位置，可以通过指针来访问该位置中存储的数据。
 - 在C/C++中，我们可以通过声明指针变量并赋予它一个地址来创建一个指针。
 - 同时，我们也可以通过解引用操作符*来访问指针指向的变量的值，或者通过给指针变量赋值来改变变量的值。

动态内存分配

- 动态内存分配是指在程序运行期间动态地申请内存空间，C/C++中通过**malloc()**函数或**new**运算符来实现动态内存分配。这种方式可以使程序在运行时按需分配所需的内存，从而实现更加灵活的内存管理。

```
#include <iostream>
using namespace std;
```

```
int main() {
    int *p; // 声明指针变量p
    p = new int; // 动态申请一个int类型的变量
    *p = 10; // 对动态申请的变量赋值
    cout << "动态申请的变量的值为: " << *p << endl; // 输出动态申请的变量的值
    delete p; // 释放动态申请的内存
    return 0;
}
```

- 上例使用了**new**运算符来动态申请了一个int类型的变量，并通过指针**p**来操作该变量，最后通过**delete**运算符来释放申请的内存。

传递参数

- 指针在函数调用中有着非常重要的作用，可以通过指针来传递参数，以达到函数间数据共享的目的。

```
#include <iostream>
using namespace std;
void swap(int *p1, int *p2) {
    int temp = *p1; // 使用指针来交换两个变量的值
    *p1 = *p2;
    *p2 = temp;
}
int main() {
    int a = 10, b = 20;
    cout << "交换前的值: a = " << a << ", b = " << b << endl;
    swap(&a, &b); // 传递变量地址来交换变量的值
    cout << "交换后的值: a = " << a << ", b = " << b << endl;
    return 0;
}
```

- 上例定义了一个swap函数，该函数接收两个指针参数，通过指针来交换两个变量的值。在主函数中，我们通过取变量地址的方式来调用swap函数，并成功地交换了两个变量的值。

使用指针访问数组

- 数组的名称本身就是数组的首元素地址，但是需要注意，其是常量指针，即不可改变的指针。
- 因此，指针还可以用来访问数组中的元素，这样可以方便地操作数组中的数据。

```
#include <iostream>
using namespace std;
int main() {
    int arr[] = {1, 2, 3, 4, 5}; // 声明一个整型数组
    int *p = arr; // 声明一个指针p，并将其指向数组首元素
    for(int i = 0; i < 5; i++) {
        cout << "arr[" << i << "] = " << *(p+i) << endl; // 使用指针来访问数组元素
    }
    return 0;
}
```

- 上例首先定义了一个包含5个元素的整型数组arr，然后定义了一个指针p，并将其指向数组的首元素。在for循环中，我们使用指针p来遍历数组中的所有元素，并输出每个元素的值。

指针算术

- 当指针指向一个数组的元素时，指针之间可以进行算术运算。指针算术的基本原则：对指针进行加减运算时，会根据指针所指向变量的类型进行对应的加减操作，即指针会自动偏移若干个字节。

```
#include <iostream>
using namespace std;
int main() {
    int arr[] = {1, 2, 3, 4, 5}; // 声明一个整型数组
    int *p = arr; // 声明一个指针p，并将其指向数组首元素
    for(int i = 0; i < 5; i++) {
        cout << "arr[" << i << "]的地址为: " << p << endl; // 输出当前指针的地址
        cout << "arr[" << i << "]的值为: " << *p << endl; // 输出当前指针指向的值
        p++; // 将指针向后移动一个位置
    }
    return 0;
}
```

- 定义了一个整型数组arr，并声明了一个指针p并将其指向数组的首元素。在for循环中，我们使用指针p来访问数组元素，并输出每个元素的值以及指针的地址，然后将指针p向后移动一个位置。

指向指针的指针

- 首先必须记住的一点，**指针也是变量，也需要内存来存储。**
- 因此，我们可以定义**指向指针的指针**，即一个指针变量存储了另一个**指针变量的地址**。指向指针的指针在某些情况下非常有用，例如在**函数中通过指针修改指针的值**等。

```
#include <iostream>
using namespace std;
int main() {
    int a = 10; // 声明一个整型变量a
    int *p1 = &a; // 声明一个指向整型变量a的指针p1，并将其初始化为a的地址
    int **p2 = &p1; // 声明一个指向指针p1的指针p2，并将其初始化为p1的地址
    cout << "a的值为: " << a << endl;
    cout << "*p1的值为: " << *p1 << endl;
    cout << "**p2的值为: " << **p2 << endl; // 通过p2来访问变量a的值
    return 0;
}
```

- 首先定义了一个整型变量a，并声明了一个指针p1来指向变量a的地址。接着，我们定义了一个指向指针p1的指针p2，并将其初始化为p1的地址。最后，我们使用指针p2来访问

const指针和指向常量的指针

- 在 C/C++ 中，我们可以使用 **const** 限制一个变量或者指针的值不能被修改，从而保护变量或者指针的值不被误操作更改。
- 当我们使用 **const** 关键字修饰一个指针变量时，我们可以通过这个指针来访问变量的值，但是不能通过这个指针来修改变量的值。因此，这个指针被称为指向常量的指针。我们可以通过以下方式来声明一个指向常量的指针：

`const int* ptr; //指向整型常量的指针`

- 这里的 **const** 关键字表示**指针所指向的内存中的值是常量**，注意是指向的内存中的值，不是指针本身，不能被修改。
- 指向的内存中的值不可以更改，但是指针的值可以更改。

const指针和指向常量的指针

- 我们还可以使用指针本身是常量的方式来定义一个指向常量的指针，也就是不能通过这个指针来修改指向的内存地址。例如：

```
int num = 10;
```

```
int* const ptr = &num; //指针本身是常量，指向整型变量 num
```

- 这里的 `const` 关键字表示指针本身是常量，不能被修改。
- 指针常量的值不可更改，但指向的内存地址可以更改。例如：

```
int num1 = 10, num2 = 20;
```

```
int* const ptr = &num1; //指针指向整型变量 num1
```

```
ptr = &num2; //错误，指针的值不能被修改
```

```
*ptr = 30; //正确，指针指向的内存地址 num1 的值被修改为 30
```

- 指针 `ptr` 被声明为指针常量，它的值不能被更改，一旦指向了 `num1`，就不能指向其他变量。但是，我们可以通过解引用操作符来修改指针所指向的内存地址中的值。

内存泄露

- 在使用指针时，一个常见的问题是内存泄漏。内存泄漏是指在程序运行期间，已分配的内存没有被释放，造成内存资源的浪费和程序性能的下降。内存泄漏通常发生在以下情况下：
 - 程序员忘记释放内存；
 - 程序异常终止，导致内存没有被释放；
 - 内存泄漏发生在循环中，导致内存逐渐耗尽。

```
#include <stdlib.h>
```

```
int main() {  
    int *ptr = (int*)malloc(sizeof(int));  
    *ptr = 10;  
    return 0;  
}
```

- 上例中就存在一个int的4字节内存没有释放。

new和delete

- new和delete操作符用于动态分配和释放内存。
- new操作符用于分配内存，并返回指向该内存块的指针，delete操作符用于释放已分配的内存。
- new和delete必须成对使用，在使用过程中，必须落实，谁申请谁释放的原则，如果谁new了一个对象，必须由new的那个人来delete，所有人都只管自己new的对象。

```
int main()
{
    // Allocate memory for an integer
    int *p = new int;
    // Assign a value to the integer
    *p = 42;
    // Print the value of the integer
    cout << *p << endl;
    // Free the memory allocated for the integer
    delete p;
    return 0;
}
```

new和delete

- 实际应用中，我们可能需要动态分配一些复杂的数据结构，例如数组或对象。下面是一个使用new和delete来动态分配和释放一个整数数组的例子：

```
int main()
{
    int *p = new int[10]; // Allocate memory for an array of integers
    for (int i = 0; i < 10; i++) // Assign values to the elements of the array
    {
        p[i] = i;
    }
    for (int i = 0; i < 10; i++) // Print the values of the elements of the array
    {
        cout << p[i] << " ";
    }
    cout << endl;
    delete[] p; // Free the memory allocated for the array
    return 0;
}
```

链表的基本概念

- 此前我们已经知道了数组是一种线性的结构，其由相同的元素组成的一系列的值。
- 与之对应的链表是一种线性数据结构，它由一系列节点构成，每个节点包含数据和一个指向下一个节点的指针。
- 链表中的节点可以在内存中任意位置，而不像数组一样需要连续的内存空间。

链表节点的结构

- 链表中的每个节点由两个部分组成：一个是数据域，用于存储节点的数据，另一个是指针域，用于存储下一个节点的地址。链表节点的结构可以用C++代码表示为：

```
template<typename T>
struct ListNode {
    T val; // 数据域
    ListNode<T> *next; // 指针域，指向下一个节点
    ListNode(T x) : val(x), next(NULL) {} // 构造函数
};
```

- 其中，T是节点存储的数据类型，val表示节点存储的数据，next是指向下一个节点的指针，NULL表示空指针。

链表的问题

- 那么链表的数据内容必须一致吗？
- 链表的地址空间必须连续或者不连续的吗？
- 链表相比于数组的优势和劣势分别是什么？

单向链表

- 单向链表，指的是有开始和结束位置，并且只有一个指向后继或者前驱指针的链表。
- 单向链表的基本操作包括遍历、插入、删除和反转等操作。
- 链表的遍历操作是指依次访问链表中的每个节点，可以使用一个指针从头节点开始依次访问下一个节点，直到指针为空（即遍历到了链表的末尾）。链表的遍历操作的时间复杂度为 $O(n)$ ，其中 n 是链表的长度。

```
template<typename T>
void traverseList(ListNode<T> *head) {
    ListNode<T> *p = head;
    while (p != NULL) {
        cout << p->val << " ";
        p = p->next;
    }
    cout << endl;
}
```

单向链表

- 链表的插入操作是指在链表中插入一个新节点，可以在任意位置进行插入操作。链表的删除操作是指从链表中删除一个节点，同样可以在任意位置进行删除操作。

```
struct ListNode {  
    int val;  
    ListNode* next;  
    ListNode(int x) : val(x), next(NULL) {}  
};
```

```
ListNode* insertAtHead(ListNode* head, int val) {  
    ListNode* newNode = new ListNode(val);  
    newNode->next = head;  
    return newNode;  
}
```

```
ListNode* deleteNode(ListNode* head, int val) {  
    if (head == NULL) {  
        return NULL;  
    }  
    if (head->val == val) {  
        return head->next;  
    }  
    ListNode* cur = head;  
    while (cur->next != NULL && cur->next->val != val) {  
        cur = cur->next;  
    }  
    if (cur->next != NULL) {  
        cur->next = cur->next->next;  
    }  
    return head;  
}
```

双向链表

- 双向链表是一种每个节点都有前驱和后继指针的链表，相比单向链表，双向链表可以双向遍历，删除节点时无需遍历查找前驱节点。双向链表的节点结构通常包括当前节点的数据域、指向前驱节点的指针prev和指向后继节点的指针next。

```
struct ListNode {  
    int val;  
    ListNode *prev;  
    ListNode *next;  
    ListNode(int x) : val(x), prev(nullptr), next(nullptr) {}  
};
```

- 双向链表的操作与单向链表类似，但需要同时维护前驱和后继指针。

双向链表

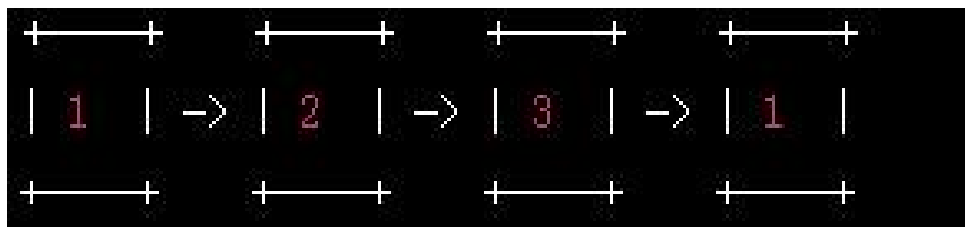
- 双向链表的插入和删除操作与单向链表类似，但需要同时维护前驱和后继指针。

```
// 在当前节点后插入一个节点
void insert(ListNode* cur, int val) {
    ListNode* newNode = new ListNode(val);
    newNode->prev = cur;
    newNode->next = cur->next;
    cur->next->prev = newNode;
    cur->next = newNode;
}

// 删除当前节点
void remove(ListNode* cur) {
    cur->prev->next = cur->next;
    cur->next->prev = cur->prev;
    delete cur;
}
```

循环链表

- 循环链表是一种特殊的链表，它的尾节点指向头节点，形成一个环。循环链表可以用于解决某些特殊问题，如环形缓冲区。



带头结点的链表

- 带头结点的链表是在**头节点上增加一个不存储数据的节点**，目的是为了简化链表的插入和删除操作。

分支语句-switch语句

Code

```
26 void switch_test()
27 {
28     float t, tc, tf; char corf;
29     const float fac = 1.8, inc = 32.0;
30     cout << "Enter temperature:";
31     cin >> t;    // 输入温度值
32     cin >> corf; // 指出是摄氏(c)或华氏(f)
33     switch(corf){
34     case 'c':
35     case 'C': tc = t; tf = t * fac + inc; break;
36     case 'f':
37     case 'F': tf = t; tc = (t - inc) / fac; break;
38     default:
39         tc = tf = 0.0;
40     }
41     cout << "The temperature is:";
42     cout << tc << "C=" << tf << "F\n";
43 }
```

Run

```
Enter temperature:43c
The temperature is:43C=109.4F
```

思考题

- 什么是地址，什么是地址中的内容，两者的区别是什么？
- 运算符“&”和“*”各有几种用法？各自的使用方式以及使用含义是什么？
- 什么是指针？地址和指针有什么样的关系？
- 指针的值和类型是怎样规定的？指针的值与其他类型变量的值是否有区别？
- 指针可以进行哪些运算？和普通数据类型的运算有什么不同？
- `void main (int argc, char * argv[]) {...}` 如何细致地描述参数 `argv` 所表达的数据结构？
- 试述指针在函数的参数传递中的作用及其使用方法。指针参数与数组参数是否有某种形式的关联？
- 什么是引用？引用和指针的区别是什么？引用型参数具有哪些优点？
- 引用型的函数返回值与非引用型的函数返回值是否有某些不相同？

思考题

- C++语言为函数间的数据传递提供了哪几种机制？从两种数据传递方向来看（从调用它的函数接收数据，“由上向下”；向调用它的函数传回数据，“由下向上”），每种机制的传递方式有什么不同？是单方向的还是双向的？
- 作用域和生存期的概念是否相同？是否有这样的变量，它在其作用域之外仍然保持占有其存储空间并保持其原有的值？
- C++程序中的名字（标识符）的作用域有哪几个等级？

思考题

- 变魔术，让计算机来猜测用户“暗记”的某张扑克牌：
 - 计算机从一副扑克牌（54 张）中任意抽出27 张，摆放在不同的 3 行上（每行 9 张）；
 - 用户“暗记”某张纸牌，而后告诉计算机所“暗记”的那张纸牌处于哪一行中；
 - 之后计算机再将纸牌重新摆放两次，并让用户再回答两次相同的提问（那张纸牌在重新摆放后又处在哪一行上）；
 - 两次之后计算机会将用户所“暗记”的那张纸牌给挑出来。